Lecture 8
Rewriting History: Rebase and Cherry-Pick

Replaying commits, tidying a branch, copying a single commit, and the one rule you must never break

Learning goals

- Explain what `git rebase` does, and how it differs from `git merge`.
- State the golden rule of rebasing and say why it exists.
- Clean up a messy branch with an interactive rebase (`fixup`, `reword`, `drop`).
- Push rewritten history safely with `--force-with-lease`.
- Copy a single commit to another branch with `git cherry-pick`.
- Resolve a conflict in the middle of a rebase.

1 Why rewrite history?

Real work is messy. You commit "wip" at five o'clock, "fix typo" ten minutes later, and "oops" after that. Before others read your branch you would like a clean story: one commit per idea, sensible messages, and a history that ends on top of the latest `main` instead of forking off an old one.

Git lets you *rewrite* history for that purpose. In Lecture 3 you already did a tiny version (`git commit --amend`). This lecture adds the big tools. They are powerful and, used wrongly, they cause exactly the confusion that Lecture 6 warned about, so we learn the safety rule together with the tools.

We use a practice repository again, with a "server" folder that plays the role of GitHub. From the course folder:

```
$ bash codeHints/L08/setup_demo8.sh
$ cd practice/demo8
$ git log --oneline --graph --all --decorate
* e04fe10 (HEAD -> main, origin/main, origin/HEAD) Add muffin recipe
| * 1f323f7 (origin/syrup) Pancakes with syrup
|/
| * 889b766 (origin/sale) Add sale banner
|/
| * 336710f (origin/cookies) fix typo in cookies
| * a7dfee0 wip
| * 45b2a27 Add cookie recipe
|/
* 699939b Add recipes
* 89ee60b Add README
```

The branch `cookies` has three messy commits and forked from `Add recipes`; meanwhile `main` moved on by one commit. The branches `sale` and `syrup` are for later. (The IDs of the starting commits match yours; the IDs of commits that *you* create below will differ, as always.) Set your identity once in this clone if you have not set it globally: `git config user.name` and `git config user.email`.

2 Rebase: replaying commits on a new base

You know one way to bring `cookies` up to date with `main`: merge `main` into it (Lecture 5). **Rebase** does it differently. It takes your commits, one at a time, and *replays* them on top of the new tip of `main`, as if you had started your work from there.

```
$ git switch cookies
branch 'cookies' set up to track 'origin/cookies'.
$ git rebase main
Successfully rebased and updated refs/heads/cookies.
$ git log --oneline --graph --decorate -5
* 5bfcea2 (HEAD -> cookies) fix typo in cookies
* dd806cd wip
* 539c33d Add cookie recipe
* e04fe10 (origin/main, origin/HEAD, main) Add muffin recipe
* 699939b Add recipes
```

(Git also prints progress lines such as `Rebasing (2/3)`, left out here.) The history is now one straight line: no fork, no merge commit.

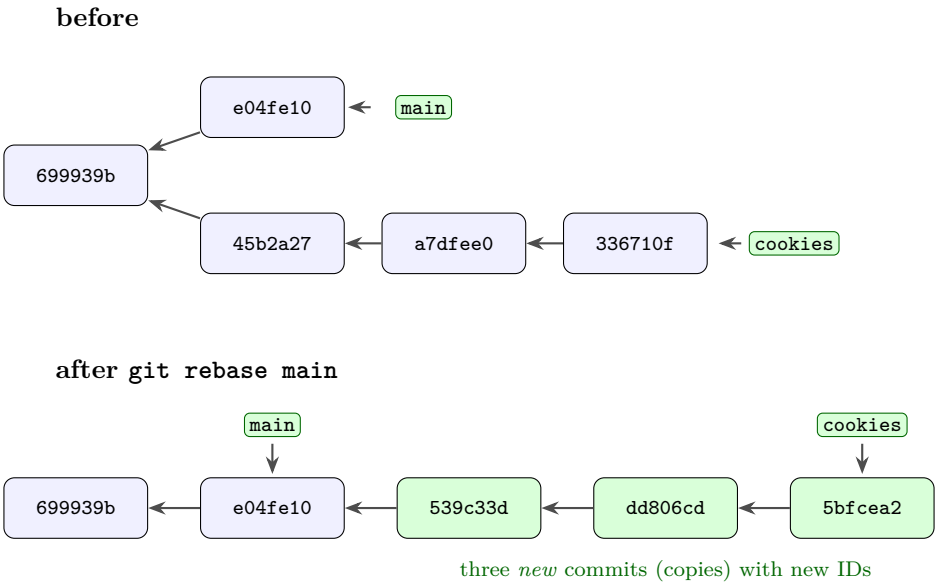


Figure 1: Rebase replays the three commits on top of `main`. The copies have the same changes and messages but *new IDs*, because their parent is different. The old commits (`45b2a27` and so on) are left behind.

Key idea

A commit's ID depends on its parent. If you move a commit onto a different parent, you get a *different commit*: same change, same message, new ID. Rebase therefore does not move commits; it **creates copies** and abandons the old ones. That is what "rewriting history" means.

2.1 Merge or rebase?

	`git merge main`	`git rebase main`
History shape	keeps the fork, adds a merge commit	one straight line
Existing commits	untouched	copied, with new IDs
Conflicts	resolved once, in the merge	possibly once per replayed commit
Safe on shared branches	yes	**no** (see the golden rule)

Neither is "right". Merge records exactly what happened. Rebase gives a tidier story. Many teams use rebase for their *own* work in progress and merge for the finished result.

3 Conflicts during a rebase

Because rebase replays commits one by one, a conflict can appear in the middle. Rebase the branch `syrup`: it edits the same line that `main`'s newest commit sits right next to.

```
$ git switch syrup
$ git rebase main
Auto-merging recipes.txt
CONFLICT (content): Merge conflict in recipes.txt
error: could not apply 1f323f7... Pancakes with syrup
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
$ cat recipes.txt
<<<<<<< HEAD
Pancakes: flour, milk, eggs
Muffins: flour, milk
=======
Pancakes: flour, milk, eggs, syrup
>>>>>>> 1f323f7 (Pancakes with syrup)
```

The rebase is paused, halfway, exactly like a conflicted merge. The procedure is the one from Lecture 5, with a different last step:

1. Edit the file: choose or combine, remove the three marker lines.
2. `git add recipes.txt` ("resolved").
3. `git rebase --continue` (not `git commit`). Git then replays the next commit, if any.

```
$ printf 'Pancakes: flour, milk, eggs, syrup\nMuffins: flour, milk\n' > recipes.txt
$ git add recipes.txt
$ git rebase --continue
Successfully rebased and updated refs/heads/syrup.
```

Pitfall

The two sides are swapped compared with a merge. During a rebase, the part above `=======` (labelled `HEAD`) is the branch you are rebasing *onto*, here `main`. The part below is *your own commit* being replayed. Read the labels, not the order, and keep what you want.

Your three escape hatches while a rebase is paused:

Command	Effect
`git rebase --continue`	After resolving: go on with the next commit.
`git rebase --abort`	Cancel everything. The branch is exactly as before you started. Always safe.
`git rebase --skip`	Drop the commit that conflicts and go on (its change is lost).

4 The golden rule

Key idea

Never rebase commits that other people already have. Rebase only commits that exist solely on your computer, or on a branch that only you use.

Why? Rebasing replaces commits with copies. If a teammate has the *old* commits and you publish the *copies*, the two of you now disagree about what the history is:

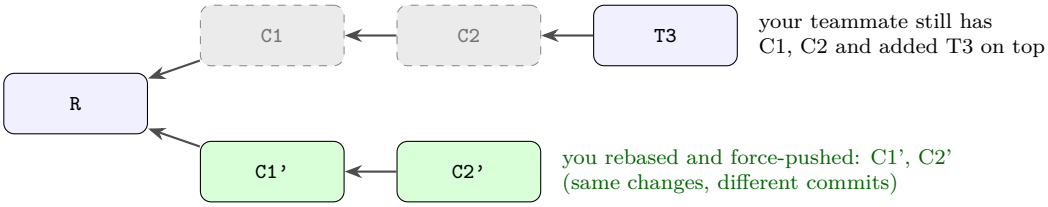


Figure 2: The teammate's `T3` sits on the old `C2`, which no longer exists on the server. Combining the two histories duplicates `C1` and `C2` and invites conflicts that nobody caused.

Commits you have only locally are safe to rewrite, and so is a branch that you alone work on, even if you pushed it, *as long as you are sure no one else has fetched it.* Never rewrite `main` or a branch your team shares.

4.1 Pushing rewritten history

After rebasing `cookies`, your local branch has copies and the server still has the originals. A normal push is refused, because it would not just add commits, it would replace them:

```
$ git push
To github.com:you/recipes.git
 ! [rejected]        cookies -> cookies (non-fast-forward)
error: failed to push some refs to 'github.com:you/recipes.git'
hint: Updates were rejected because the tip of your current branch is behind
```

This is the situation for which beginners reach for `git push --force`, which overwrites the remote branch *unconditionally.* Use the safer cousin instead:

```
$ git push --force-with-lease
To github.com:you/recipes.git
 + 336710f...8d96fcc cookies -> cookies (forced update)
```

`--force-with-lease` means "replace the remote branch, but *only if it is still where I last saw it*". If someone pushed in the meantime, it refuses, and you find out before you destroy their work:

```
$ git push --force-with-lease
```

```
 ! [rejected]        sale -> sale (stale info)
```

(Here a friend had pushed a commit to `sale` after your last `git fetch`.) When you see `stale info`, fetch, look at what arrived, and decide; do not escalate to `--force`.

Pitfall

Even `--force-with-lease` must not be used on `main` or any shared branch. Hosting sites can forbid it with *branch protection*; Lecture 9 shows how.

5 Interactive rebase: tidying your commits

The most useful form of rebase lets you *edit the list of commits*. Our `cookies` branch has the story "Add cookie recipe, wip, fix typo". We want a single commit. Start an interactive rebase onto `main`:

```
$ git rebase -i main
```

(You may also say `git rebase -i HEAD~3`: "the last three commits".) Git opens your editor with a **todo list**, oldest commit first, one command per line:

```
pick 539c33d Add cookie recipe
pick dd806cd wip
pick 5bfcea2 fix typo in cookies

# Rebase e04fe10..5bfcea2 onto e04fe10 (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup <commit> = like "squash" but keep only the previous commit's message
# d, drop <commit> = remove commit
# (...more commands and a warning about deleted lines...)
```

Change the word at the start of a line to tell Git what to do with that commit. To fold the second and third commit into the first, replace `pick` by `fixup` on those lines, then save and close the editor:

```
pick 539c33d Add cookie recipe
fixup dd806cd wip
fixup 5bfcea2 fix typo in cookies
```

```
$ git log --oneline --decorate -4
8d96fcc (HEAD -> cookies) Add cookie recipe
e04fe10 (origin/main, origin/HEAD, main) Add muffin recipe
699939b Add recipes
89ee60b Add README
```

Three commits became one, with the combined changes and the first message.

Command	What it does
`pick`	Keep the commit as it is.
`reword`	Keep the changes, but let me edit the message (the editor opens again).
`squash`	Merge into the previous commit and let me combine both messages.
`fixup`	Merge into the previous commit and discard this message.
`drop`	Delete the commit (or delete the line). Its changes are gone.
`edit`	Stop after this commit so I can change it (then `git rebase --continue`).

You can also *reorder* commits by moving lines. The commits are replayed from top to bottom. To change your mind at any moment before you finish: `git rebase --abort`.

Pitfall

The editor. If Git opens `vim` and you do not know it: press `i` to start typing, edit the text, press `Esc`, then type `:wq` and press Enter to save and quit (`:q!` quits without saving, which cancels the rebase). Or choose an editor you know, for example `git config --global core.editor "nano"`.

Quick fact

Shortcut for the future: `git commit --fixup <id>` makes a commit that is marked as a fix for an earlier commit, and `git rebase -i --autosquash main` then arranges and folds it automatically. You do not need it now.

6 Cherry-pick: copying one commit

Sometimes you want *one* commit from another branch, not the whole branch. The branch `sale` has a commit `Add sale banner`, and `main` should get it now:

```
$ git switch main
$ git cherry-pick 889b766
[main 26e9c4f] Add sale banner
 Author: Practice User <practice@example.com>
 Date: Wed Nov 6 09:00:00 2024 +0000
 1 file changed, 1 insertion(+)
 create mode 100644 banner.txt
```

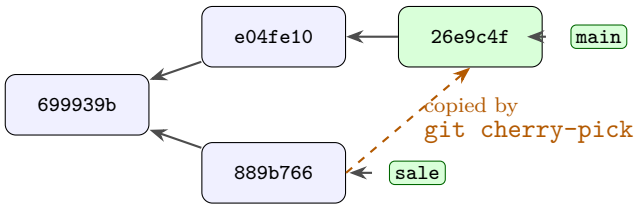


Figure 3: Cherry-pick creates a *copy* of `889b766` on top of `main`. The original stays on `sale`.

The copy has a new ID and parent, but the same change, message, author and original date. If the change does not apply cleanly you get a conflict, which you resolve as before, then finish with `git cherry-pick --continue` (or cancel with `git cherry-pick --abort`). Adding `-x` appends "(cherry picked from commit ...)" to the message, which helps later.

Typical uses:

- **A hotfix** made on one branch that must also go to `main`.

- **A commit made on the wrong branch** (the mistake warned about in Lecture 4): switch to the right branch and cherry-pick the commit there. The copy on the wrong branch must still be removed; that needs `git reset`, introduced in Lecture 10.

Pitfall

Cherry-pick *duplicates* a change. If you later merge the whole branch as well, Git usually recognises the same change and copes, but long-lived copies cause confusing conflicts. Use it for single, deliberate commits, not as a way to move branches around.

7 Pulling with rebase

In Lecture 6 a push was rejected because the server had a commit you lacked, and `git pull` asked you how to reconcile. Merge was one answer. **Rebase** is the other: put your unpushed commits *on top of* what the server has.

```
$ git push
 ! [rejected]        main -> main (fetch first)
$ git pull --rebase
Successfully rebased and updated refs/heads/main.
$ git log --oneline --graph --decorate -4
* e3126f3 (HEAD -> main) Add local note
* 418c700 (origin/main, origin/HEAD) Add friend news
* e04fe10 Add muffin recipe
* 699939b Add recipes
$ git push
   418c700..e3126f3  main -> main
```

The result is a straight line with no merge commit. This is safe, because your local commit had not been shared yet, so replacing it by a copy hurts nobody. You can make it the default with `git config --global pull.rebase true`; the choice you made in Lecture 6 (`false`, merge) was equally valid.

8 Rebase and GitHub

Lecture 7 offered three merge buttons. You can now read two of them: **Squash and merge** is the `fixup` of this lecture done by GitHub, and **Rebase and merge** is a rebase done by GitHub (new commits again, hence the `-d` surprise).

A pull request that has fallen behind `main` can be brought up to date by merging `main` into it (always safe) or by rebasing it and pushing with `--force-with-lease`. Choose the rebase *only* when you are the sole author of the branch.

Quick fact

Rebased commits are not destroyed at once. Git remembers where your branch was before, so even a disastrous rebase can usually be undone. Lecture 10 shows how with the *reflog*. Until then, `git rebase --abort` is your friend during a rebase.

9 Homework 8

Homework 8: Clean up and ship

The setup script creates `practice/hw8_server.git`, a shared "server" repository with three branches: `main`, `feature` (five messy commits, already pushed) and `hotfix` (one commit). From the course folder:

```
$ bash codeHints/L08/setup_hw8.sh
$ cd practice
```

You are the only person working on `feature`, so rewriting it is allowed. Do the tasks in order. To start over: `setup_hw8.sh --reset` from the course folder.

8.1 Look

Clone the server into `hw8`, go in, and run `git log --oneline --graph --all --decorate`.

8.2 Tidy and rebase

On `feature` run `git rebase -i main`. Turn the five commits into *two*: fold the `wip` and `fix typo` commits into `Add greeting`, fold `oops` into `add farewell`, and *reword* `add farewell` to `Add farewell`. The first commit conflicts in `README.md`: keep *both* added lines (`Maintained by the team.` and `Usage: run the greeter.`), remove the markers, `git add README.md`, and `git rebase --continue`.

8.3 Check

`git log --oneline main..feature` must show exactly two commits, `Add greeting` and `Add farewell` (oldest first), on top of `main`'s newest commit.

8.4 Publish

Try a plain `git push` and read the rejection. Then push with `git push --force-with-lease`.

8.5 Cherry-pick

Switch to `main`. Find the commit on `origin/hotfix` (`git log origin/hotfix`) and copy it onto `main` with `git cherry-pick`. Push `main`. (`main` must stay a straight line: no merge.)

8.6 Questions

In `practice/hw8_answers.txt` (next to `hw8`, created with `printf` or `echo`) write three lines: `shared-rebase=<yes or no>` (is it fine to rebase commits your teammates have already pulled?), `safer-force=<option>` (the safer alternative to `--force`), and `new-ids=<merge or rebase>` (which of the two creates copies of your commits with new IDs?).

Check yourself: from the course folder run `bash codeHints/L08/check_hw8.sh`. All `PASS`, then compare with the solutions at the end of this lecture (or in `solutions/L08/`).

10 Summary

- Rebase replays your commits on a new base, creating copies with new IDs: a straight history instead of a fork. A rebase conflict is resolved with `add` and `git rebase --continue`; `--abort` undoes it all.
- Golden rule: never rebase commits others already have. After rebasing your own pushed branch, use `--force-with-lease`, never plain `--force`.
- `git rebase -i`: `pick`, `reword`, `squash`, `fixup`, `drop`, reorder.
- `git cherry-pick` copies one commit onto the current branch. `git pull --rebase` avoids a merge commit when you only have local commits. (*Pro Git*, Chapters 3.6, 7.6.)

Solutions

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

Solutions to Homework 8

```
$ bash codeHints/L08/setup_hw8.sh
$ cd practice
$ git clone hw8_server.git hw8
$ cd hw8
$ git log --oneline --graph --all --decorate
$ git switch feature
$ git rebase -i main
```

The editor opens with the five commits, oldest first (your IDs differ from here on):

```
pick 74e9302 Add greeting
pick b8385c7 wip
pick c59f243 fix typo
pick 3b4d390 add farewell
pick 83ae7a6 oops
```

Change the first word of lines 2 to 5, save and close:

```
pick 74e9302 Add greeting
fixup b8385c7 wip
fixup c59f243 fix typo
reword 3b4d390 add farewell
fixup 83ae7a6 oops
```

Git replays `Add greeting` first and stops with a conflict in `README.md`, because `main`'s newest commit also added a line there. In the file, the part above `=======` is `main`'s line and the part below is yours; keep both:

```
# Greeter
Maintained by the team.
Usage: run the greeter.
```

```
$ git add README.md
$ git rebase --continue          # an editor opens for the reword step:
                                 # change "add farewell" to "Add farewell", save, close
$ git log --oneline main..feature
$ git push                       # rejected (non-fast-forward): expected
$ git push --force-with-lease
$ git log --oneline origin/hotfix -1
$ git switch main
$ git cherry-pick <that-id>
$ git push
$ printf 'shared-rebase=no\nsafer-force=force-with-lease\nnew-ids=rebase\n' > ../
    hw8_answers.txt
```

8.2. `fixup` merges a commit into the one above and keeps the *upper* message, so `wip` and `fix typo` disappear into `Add greeting`, and `oops` into `Add farewell`. `reword` stops once more so that you can fix the capital letter. If you used `squash` instead of `fixup`, Git opens an editor with combined

messages, and you must delete the extra ones, otherwise the checker finds the wrong messages.

8.3. Two commits, `Add greeting` and `Add farewell`, on top of `Update README`. If `git log --oneline main..feature` shows five commits, you ran a plain rebase without `-i` or forgot to edit the list.

8.4. The rebase created copies of the commits, so the server's `feature` (with the old five) is no longer an ancestor of yours, and a normal push is rejected. `--force-with-lease` is allowed to replace it because the server's branch is exactly where you last saw it.

8.5. `git cherry-pick` copies the hotfix commit onto `main`: a *new* commit with the same change and message. The original stays on `hotfix`. `main` remains a straight line of four commits. (Merging `hotfix` would add the original commit and a merge commit, which the checker rejects.)

8.6. `shared-rebase=no` (golden rule), `safer-force=force-with-lease`, `new-ids=rebase` (merge keeps all existing commits and only adds one).

Common mistakes.

- Merging `main` into `feature` instead of rebasing: the history keeps five messy commits plus a merge commit.
- Leaving the rebase unfinished (after a conflict you must `git add` and `git rebase --continue`). `git status` says `rebase in progress`; the checker's last check fails until you finish or `git rebase --abort`.
- Using plain `git push --force`. It works here, but it is exactly the habit the lecture warns against; use `--force-with-lease`.
- Stuck in the vim editor: press `Esc`, type `:wq`, press Enter. To cancel the whole rebase instead, type `:q!` on an empty todo list, or run `git rebase --abort`.